T1T5-Test Design - Scenariobased Testing (T1..T5)

- Scenario Testing
- The name “T1T5 Test Design”
- Title Syntax
  - Example 1 - Paying the parking fee
  - Positive Case
  - Negative Case
  - Example 2 - Login to application
- Reason
  - Test name should express a specific requirement
  - Test name should include the expected input or state and the expected result for that input or state
  - Test name should be presented as a statement or fact of life that expresses workflows and outputs

Scenario Testing

- is a Software Testing Technique that uses scenarios i.e. speculative stories or situations to help the tester work through a complicated problem or test system. The ideal scenario test is a reliable, complicated, convincing or motivating story the outcome of which is easy to assess.
- Scenario testing is performed to ensure that the end to end functioning of software and all the process flow of the software are working properly. In scenario testing, the testers assume themselves to be the end users and find the real world scenarios or use cases which can be carried out on the software by the end user. In scenario testing, the testers take help from clients, stakeholders and developers to create test scenarios.
- Scenario testing helps testers to know how the software will exactly work when end user will use it. As the scenario testing tests the business process flow of the software so it helps in figure out a lot of defects which cannot be found with the help of other testing. Scenario testing is carried out by creating test scenarios which copy the end users usage. A test scenario is a story which describes the usage of the software by an end user.

The name “T1T5 Test Design”

- for scenario based testing was coined during the RBI Group Digital Solution program as the outcome of a test design workshop. The term T1..T5 is based on the scenario cards from MaibornWolff (https://github.com/MaibornWolff/scenario-cards/tree/main/Spielkarten/English - Connect your Github account). In the beginning of GDS, the name was only used by the GDS-Newton team, but with the implementation of the GDS Testing Chapter all other GDS-Projects took over the name T1..T5 and established it as common term.

Title Syntax

We are using a special syntax for the title of the test scenario:

- <2 character abbreviation>_<Feature>_<BusinessRule|Requirement|>_<Expected_Result>
- we value
  - being able to understand what behavior is being tested by reading just the test name
  - knowing the scenario and the expected outcome by reading the name
  - being able to easily tell if some functionality isn't tested
  - seeing what functionality is broken without looking at the test's source code or detailed description
  - seeing which scenario is broken without looking at the test's source code or detailed description
- how
  - Test name should express a specific requirement
  - Test name should include the expected input or state and the expected result for that input or state
  - Test name should be presented as a statement or fact of life that expresses workflows and outputs

- Test Name should begin with the 2 character abbreviation of the covered scenario

Example 1 - Paying the parking fee

Positive Case	aka	Description	Example	Comment
Standard Case	T1	The standard case represents the favourite sequence on interactions with the system in order to achieve the desired application goals.	• Payment in cash • coins 10, 20, 50 cent, 1, 2 euro • exact amount	Also called "Happy Path"
Alternative Case	T2	The alternative case pursues the same application goals as the standard case but describes an alternative sequence of interactions with the system.	• payment with EC-Card • valid card, PIN ok	
Exception Case	T3	The exception case pursues the same application goals as the standard case and the alternative case. The exception describes a sequence of interactions with the system that should be executed on deviations from to the standard or alternative case.	• payment in cash, not exact amount, too much • notice and return of money (excess amount)	

Negative Case	aka	Description	Example	Comment
Negative Case	T4	The negative case consciously does not pursue the application goals of the standard, alternative and exception cases (so-called happy cases). In the negative case, invalid test data is used within the same sequences of interactions as in the happy cases to check that they a accurately rejected by the system.	• cash payment, with 5 cent coin (unauthorised) • coin is ignored / falls through	
Misuse Case	T5	The misuse case describes an undesired sequence of interactions with the system to assess how robust the system is and whether it is possible to manipulate or outsmart it.	• payment with an iron button • button is ignored / falls through	

Example 2 - Login to application

Feature

A user with valid credentials (username+password) can login to the application.

Test Scenario Names

- Standard Case
  - T1_*login_validUserWithCorrectPassword_userIsLoggedInAndGetsToken*
- Alternative Case
  - T2_*login_validUserWithValidIdCard_userIsLoggedInAndGetsToken*
- Exception Case

    - T3_*login_validUserWithNotReadableIdCard_userGetErrormessageAndIsNotLoggedIN*
- Negative Case
    - T4_*login_validUserWithWrongPassword_userGetErrormessageAndIsNotLoggedIN*
- Misuse Case
    - T5_*login_validUserWithLockedIdCard_userGetErrormessageAndIsNotLoggedIN*

Reason

Test name should express a specific requirement

Your test name should express a specific requirement. This requirement should be somehow derived from either a business requirement or a technical requirement. In any case, that requirement has been broken down into small enough pieces, each of which represents a test. If your test is not representing a requirement, why are you writing it? Why is that code even there?

Test name should include the expected input or state and the expected result for that input or state

To express the specific requirement clearly you should think about two things you need to express: the input values/current state during which the test takes place, and the expected result from that state.

Test name should be presented as a statement or fact of life that expresses workflows and outputs

Unit tests are just as much about documentation as they are about testing functionality. You should always assume that someone else is going to read your tests in a year, and that they should not have a hard time understanding your intent.

Test names which are written as short declarative statements are much more expressive that names which are machine readable that is they only make sense if you have an understanding of a hidden code language to dissect their meaning.